

Review: if, else and boolean expressions (solution)

COSC 1210 (CS I) Fall 26 | Augustana U. | Monday 2026-09-28

Questions and solutions

1. You want to check whether $2 + 2$ is 4. Python refuses to run this:

```
print(2 + 2 = 4)
```

Why does it fail? Write the corrected line.

Solution. SyntaxError: = **assigns** a value to a name, but a comparison needs ==, which **compares** two values and gives True or False.

```
print(2 + 2 == 4)
```

True

2. **Logical operators.** What does this print?

```
print(5 > 3 and 2 > 4)
print(5 > 3 or 2 > 4)
print(not (3 == 3))
```

Solution.

```
print(5 > 3 and 2 > 4)
print(5 > 3 or 2 > 4)
print(not (3 == 3))
```

False
True
False

3. **Exclusive or.** What does this print?

```
a = True
b = False
print(a ^ b)
```

Solution. ^ is the bitwise xor (not a shift): $a \text{ xor } b$ is true only if a and b have different boolean values.

```
a = True
b = False
```

```
print(a ^ b)
```

True

4. **Between two values.** You want to check whether the fuel level is between 90 and 100 (both included). Python refuses to run this:

```
fuel = 96
print(fuel >= 90 and < 100)
```

Write the corrected line.

Solution. `SyntaxError`: each side of `and` must be a complete comparison, and `< 100` has no left-hand side. Repeat the name, and use `<=` to include the 100.

```
fuel = 96
print(fuel >= 90 and fuel <= 100)
```

True

5. **Integer division and modulus.** The clock should print `T-2:05` for 125 seconds, but it prints `T-5:02`. Which two operators have to change, and to what?

```
def clock(seconds):
    minutes = seconds % 60
    secs = seconds // 60
    print(f"T-{{minutes}}:{{secs:02d}}")

clock(125)
```

Solution. The operators are swapped: `//` (floor division) gives the whole minutes (`125 // 60 = 2`), `%` (modulus) gives the seconds left over (`125 % 60 = 5`).

```
def clock(seconds):
    minutes = seconds // 60
    secs = seconds % 60
    print(f"T-{{minutes}}:{{secs:02d}}")

clock(125)
```

T-2:05

6. **The if statement.** The wind station is GO when the wind speed is between 0 and 25 km/h (25 itself is too much). Fill in the blanks.

```
wind = 12
if 0 _____ wind _____ 25 _____
    print("Wind: GO")
print("(wind poll finished)")
```

What is printed for wind = 12? For wind = 25?

Solution. The blanks are <, < and : (the colon ends the header). Python allows the comparison to be chained: 0 < wind < 25. It has to be < and not <=, because 25 itself is too much. The last line is not indented, so it is not part of the if body and always runs.

```
for wind in (12, 25):
    print("wind =", wind)
    if 0 < wind < 25:
        print("Wind: GO")
    print("(wind poll finished)")
```

```
wind = 12
Wind: GO
(wind poll finished)
wind = 25
(wind poll finished)
```

7. **The else clause.** Complete the function so that it prints whether a number is even or odd.

```
def is_even(n):
    """Print whether n is even or odd."""
    if n _____ 2 _____ 0:
        print(n, "is even")
    _____
        print(n, "is odd")
is_even(7)
```

What does is_even(7) print?

Solution. The blanks are %, == and else:. A number is even if the remainder of the division by 2 is 0. Exactly one of the two bodies runs.

```
def is_even(n):
    """Print whether n is even or odd."""
    if n % 2 == 0:
        print(n, "is even")
    else:
        print(n, "is odd")
is_even(7)
```

```
7 is odd
```

8. **One statement instead of three.** All three stations must be GO (minimum: fuel 90, engines 95, guidance 99). Fill in the blanks.

```
fuel, engines, guidance = 96, 88, 100
if fuel _____ 90 _____ engines _____ 95 _____ guidance >= 99:
    print("GO")
```

```
else:
    print("NO-GO")
```

What is printed?

Solution. The blanks are `>=`, `and`, `>=` and `and`: each reading must reach its minimum, and all three conditions must be `True`. It prints `NO-GO`, because the engines reading 88 is below the minimum

1.

```
fuel, engines, guidance = 96, 88, 100

if fuel >= 90 and engines >= 95 and guidance >= 99:
    print("GO")
else:
    print("NO-GO")
```

NO-GO

9. **Refactoring.** The steps are the same for every station; only the station name, the reading and the minimum change. Fill in the blanks.

```
def go_nogo(_____, reading, minimum):
    """Print GO if the reading reaches the minimum, else NO-GO."""
    if reading _____ minimum:
        print(f"{_____}: GO")
    else:
        print(f"{system}: NO-GO")

go_nogo("Engines", 88, 95)
```

What does the call print?

Solution. The blanks are `system`, `>=` and `system`. The name of the parameter must match the name used in the f-strings.

```
def go_nogo(system, reading, minimum):
    """Print GO if the reading reaches the minimum, else NO-GO."""
    if reading >= minimum:
        print(f"{system}: GO")
    else:
        print(f"{system}: NO-GO")

go_nogo("Engines", 88, 95)
```

Engines: NO-GO

10. **Extending a function.** Change `go_nogo` into `range_check` so that a reading is `GO` if it lies between `low` and `high`, both included. Only the header and the condition change. Fill in the blanks.

```
def range_check(system, reading, _____, _____):
    """Print GO if the reading lies between low and high."""
    if reading _____ low _____ reading _____ high:
```

```
        print(f"{system}: GO")
    else:
        print(f"{system}: NO-GO")

range_check("Cabin pressure", 101, 95, 105)
range_check("Cabin pressure", 110, 95, 105)
```

What do the two calls print?

Solution. The blanks are low, high, >=, and and <=. The reading must be at least low and at most high; using >= and <= includes both ends.

```
def range_check(system, reading, low, high):
    """Print GO if the reading lies between low and high."""
    if reading >= low and reading <= high:
        print(f"{system}: GO")
    else:
        print(f"{system}: NO-GO")

range_check("Cabin pressure", 101, 95, 105)
range_check("Cabin pressure", 110, 95, 105)
```

```
Cabin pressure: GO
Cabin pressure: NO-GO
```

Author: Marcus Birkenkrahe

Created: 2026-10-02 Fri 21:22